Machine Problem 4

ECE 511: Computer Architecture (Spring 2026)

In this machine problem (MP), we will examine the performance bottlenecks of NPUs for large language model (LLM) inference workloads, and optimize the NPU architecture for efficiency improvement. We define this MP as a semi-open course project, in which we will offer concrete instructions for most of the tasks, but also encourage you to develop new thoughts through the process. We expect that you will be dedicated to working on the NPU architecture for multiple weeks until the end of this semester, with a focus on its design and optimizations for LLM inference.

The MP has five parts, as presented in the list below. In Part I, you will perform a survey of related studies to learn the technical background and state-of-the-art research on NPU architecture and the execution workflow of LLM inference workload on NPU chips. To facilitate this study, you need to collect technical papers from top-tier computer architecture conferences (such as ISCA, ASPLOS, MICRO, and HPCA), white papers released by industry, and even technical blogs. In Part II, you will start to examine the basic concepts of NPU architecture with our open-source NPU simulator NeuSim. To facilitate your understanding, you will conduct a characteristic study of NPU architecture using diverse ML workloads. In Part III, you will focus on LLM inference workloads and examine the performance bottlenecks of an NPU chip for different inference stages (i.e., prefill and decode). This study will serve as the motivation for prefill-decode (PD) disaggregation. In Part IV, you need to extend NeuSim to support the execution of PD disaggregation on NPU chips, and examine the end-to-end performance. In Part V, based on the studies of the first four tasks, you need to develop optimization techniques to improve the NPU architecture for PD disaggregation. For all written parts through MP4 development, use the ECE 511 LaTeX template [1]. You may add or remove sections in the template as needed.

Deliverables: You will deliver the following tasks by the defined milestones and deadlines.

- Part I (4%, due 3/23): Related work investigation on NPU chip design and current LLM inference paradigms.
- Part II (6%, due 4/6): Examine the basic concepts of NPU architecture with NeuSim, and conduct a characteristic study of NPU chips.
- Part III (9%, due 4/20): Examine the performance bottlenecks of an NPU chip for LLM inference.
- Part IV (8%, due 5/1): Extend NeuSim to support PD Disaggregation and examine its end-to-end performance.
- Part V (8%, due 5/15): Develop optimization techniques to improve the NPU architecture for supporting PD Disaggregation.

1 Investigation of Related Works

The rising popularity of machine learning workloads has prompted architects to develop hardware tailored to the computations found in deep neural network models such as Large Language Models (LLMs). The neural processing unit (NPU) is one popular architecture for accelerating ML workloads, exemplified by commercial products like Google TPU, Amazon Trainium, and Intel Gaudi. NPUs have become a popular choice for serving LLMs at scale.

[1] Latex template link: https://www.overleaf.com/read/gzwqdmsstksf#e965cc

We begin this MP with a related work investigation to understand NPU architecture and LLM inference workloads. Please refer to at least 10 papers, including those listed below as *required*. For the rest, you can choose any papers related to NPU architecture and/or LLM inference serving. Before you get started, we recommend familiarizing yourself with LLMs and the general properties of LLM inference if you do not have pre-existing background knowledge. The Illustrated Transformer and Mastering LLM Techniques: Inference Optimization are good starting points. When conducting your investigation, it would be helpful to keep in mind the following guiding questions: What is the core idea behind NPU architecture, and why is it well-suited to machine learning computations? How does it differ from other ML accelerator architectures (such as GPUs)? How have NPU architectures evolved? How is LLM inference performed on NPU chips? What are the techniques used to improve LLM inference?

Your submission should be approximately **1-1.5 pages** long. The **Required and recommended papers** include:

1. **[Required]** In-Datacenter Performance Analysis of a Tensor Processing Unit (Google TPUv1) [4]
2. The Design Process for Google's Training Chips: TPUv2 and TPUv3 [5]
3. **[Required]** An Optically Reconfigurable Supercomputer for ML (Google TPUv4) [3]
4. Resiliency at Scale: Managing Google's TPUv4 Machine Learning Supercomputer [9]
5. **[Required]** Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism [8]. *Note: focus on understanding tensor and pipeline parallelism.*
6. **[Required]** Splitwise: Efficient Generative LLM Inference Using Phase Splitting [6]
7. Life-Cycle Emissions of AI Hardware: A Cradle-To-Grave Approach and Generational Trends [7]

2 NPU Architecture Exploration and Device Simulation

2.1 NeuSim Installation and Overview

In this section, we will use NeuSim, a cycle-level NPU simulator, to explore the impact of NPU architectural decisions on the LLM inference performance. You should begin by installing NeuSim using the commands below (make sure to install conda or miniconda first if you haven't already):

```
git clone git@github.com:platformxlab/NeuSim.git
cd NeuSim

conda create --name neusim python=3.12.2
conda activate neusim
pip install -e .
pip install -e '.[dev]'
```

NeuSim models the key components inside a Neural Processing Unit (NPU) chip: **systolic arrays** (SAs) for matrix multiplications, **SIMD vector units** (VUs) for element-wise and reduction operations, an on-chip **SRAM scratchpad** (Vmem) for fast access to in-use data, and off-chip **high-bandwidth memory** (HBM) for storing the complete model weights and activations. Given a deep learning workload expressed as a list of operators, NeuSim iterates over the list and computes the per-operator execution statistics for each (i.e., compute time, memory time, communication time). Using this information, it determines the execution time based on the resource bottleneck for every operator.

The main entrypoint for NeuSim is the `neusim/run_scripts/run_sim.py` script. It generates a set of NPU chip configurations and model configurations with `generate_all_run_configs()`, then executes them in the `generate()` function using

an `OpsGenerator` to do the heavy lifting. The OpsGenerator creates the list of operators corresponding to the model, and uses `analysis_lib.fill_operators_execution_info` to simulate each operator's execution on the modeled NPU hardware.

Understanding the output. After a simulation completes, results are written to the `results/raw/` directory (or the directory specified by `--output_dir`). For LLM inference, NeuSim produces the following files for each configuration:

- `*_prefill.csv` and `*_decode.csv`: Per-operator traces with columns for execution time, compute time, memory time, FLOP count, bytes accessed, and which component bottlenecks the operator execution latency (`Compute`, `Memory`, or `ICI`).
- `*_prefill.json` and `*_decode.json`: End-to-end statistics, including:
  - `TTFT_sec`: Time to First Token (prefill latency in seconds).
  - `throughput_tokens_per_sec`: Aggregate token throughput.
  - `TPOT_ms_request`: Time Per Output Token per request (decode latency in ms).
  - `mem_footprint_GB`: Memory footprint of the model.
  - `out_of_memory`: Whether the configuration exceeds HBM capacity.

The output directory is organized as: `results/raw/<model>/<parallelism_string>/<workload>-v<version>.csv`. For single-chip inference experiments, the parallelism string will be `dp1-tp1-pp1-dpdcn1-tpdcn1-ppdcn1-bs<batch_size>`.

2.2 Creating a Model Ops Generator in NeuSim

NeuSim represents deep learning models as sequences of composable *operators*. An *ops generator* is a Python class that constructs this operator sequence for a specific model architecture. Each operator carries information about the tensor dimensions, associated data, and resources it uses for execution. Given an operator definition, the simulator can use `analysis_lib.fill_operators_execution_info` to determine how the operator executes on the modeled chip.

Many generators already exist in the repository (see `neusim/npusim/frontend/`). To help you get familiar with the NeuSim simulator, LLM model architecture, and NPU architecture, you need to implement a new ops generator for the GPT-OSS-20b model [1] from OpenAI. Unlike the models currently supported by the simulator, GPT-OSS models use interleaved sliding window attention [2] layers to reduce the memory footprint of queries with long sequence lengths. You will add support for this new operator as described in § 2.2.2.

To help you get started, we've provided a patch file containing skeleton code to guide your implementation. You can run the below commands in the NeuSim project root to download and apply the patch. *Note: if you encounter a "corrupt patch" error, make sure there's a newline at the end of the patch file.*

```
# Run this from the NeuSim project root -- Download patch file
curl -sL https://raw.githubusercontent.com/MichaelWang237/ece511-2026-public/main/mp4/gpt_oss.patch \
    -o gpt_oss.patch
# Apply patch file
git apply --verbose gpt_oss.patch
```

The patch provides skeleton code for both GPT-OSS-120b and GPT-OSS-20b. Both models share the same architecture (same `GptOssConfig` class, same `GptOssOpsGenerator` class); they differ only in the number of layers (24 vs. 36) and the number of MoE experts (32 vs. 128). **For the deliverables in this section, use the GPT-OSS-20b model** (`configs/models/gpt-oss-20b.json`).

2.2.1 Configuration

The patch defines several configuration objects required to implement the support for GPT-OSS models. *You do not need to do anything for the configuration; this section is just for your reference.*

1. `configs/models/gpt-oss-20b.json` – Configuration for GPT-OSS-20b (24 layers, 32 experts, top-4 routing, sliding window size 128).
2. `configs/models/gpt-oss-120b.json` – Configuration for GPT-OSS-120b (36 layers, 128 experts, same architecture otherwise).
3. `neusim/configs/models/LLMConfig.py`: `class GptOssConfig(MoELLMConfig)` – Shared config class used by the Ops-Generator for both model sizes.

2.2.2 Defining a New Operator - Sliding Window Attention

In contrast to the full attention mechanisms used by many LLMs, where each token attends to all other tokens in the sequence, sliding window attention only requires a token to attend to the previous N tokens in the sequence, where N is a constant defined by the model architecture. Many new models, including GPT-OSS, use sliding window attention for its reduced computational and memory overhead.

The sliding window attention is not yet implemented in NeuSim. You must first implement this operator before you can develop the GPT-OSS ops generator. To guide your implementation, the patch file provides the following function interface and helpful hints at line 2103 in `neusim/npusim/frontend/llm_ops_lib.py`. *Hint: the existing* `create_multi_head_attention()` *function may help simplify your implementation.*

```
def create_sliding_window_attention(
    batch_size: int,
    q_seqlen: int,          # full sequence length for Q
    sliding_window_size: int, # window size for K/V
    num_heads: int,
    num_kv_heads: int,
    d_model: int,
    d_head: int,
    num_layers: int,
    config: ModelConfig,
    dtype: str = DT_BFLOAT16,
    fusion_id_start: int = 0,
    is_decode: bool = False,
    description_prefix: str = ,
    use_flash_attention: bool = False,
    tensor_parallelism_axes: Sequence[int] = [1],
    ici_bw_GBps: float = 900.0,
) -> list[Operator]:
```

2.2.3 Defining the Ops Generator

Next, we need to create the ops generator itself by filling out the provided class at `neusim/npusim/frontend/llm_ops_generator.py`. Your job is to implement the `compute_memory_footprint_bytes`, `generate_prefill_ops` and `generate_decode_ops` methods. You may refer to existing ops generator classes in the simulator for reference, as well as the provided JSON configuration file

and the model card [1] if necessary. You may also find the following resource helpful for understanding the model architecture.[2]

As GPT-OSS is a mixture-of-experts (MoE) model, it bears some resemblance to the existing DeepSeek model that is currently supported by the simulator. For the MoE FFN of GPT-OSS, you may reuse the logic for DeepSeek MoE by specifying the type when creating the FFN operator, as shown below. Before you get started on your implementation, we recommend that you read through the walkthrough (§ 2.2.4).

```
ops_lib.create_ffn(
    ...
    num_layers=self.num_layers,
    ffn_type = 'deepseek_moe',   # reuse MoE FFN infrastructure
    ...
)
```

Important Note: for simplicity, you only need to consider the attention mechanism and the FFN for each layer. For this part of the MP, you do not need to implement support for model parallelism. Additionally, you can use BFLOAT16 as the datatype for all tensors.

2.2.4 Walkthrough: How the Llama-3-8B Ops Generator Works

To help you understand the OpsGenerator pattern before implementing your own, this section walks through the existing Llama-3-8B generator (`LLMOpsGenerator` in `llm_ops_generator.py`). A standard transformer inference layer consists of: (1) layer normalization, (2) multi-head attention, and (3) a feed-forward network (FFN). The ops generator creates operator objects for each of these components, and the simulator computes execution statistics for every operator.

Key concepts.

- **Operator**: An operator represents one computational step (e.g., a matrix multiplication, a LayerNorm, a FlashAttention kernel). Each operator has input/output tensor shapes, a type (`MXU` for systolic array ops, `VPU` for vector unit ops), and a count field indicating how many times this operator is repeated. For instance, setting `count=32` means this single operator object models the same computation repeated across 32 layers.
- **Prefill vs. Decode**: In LLM inference, the *prefill* phase processes the entire input prompt in parallel, while the *decode* phase generates one token at a time autoregressively. These have very different tensor shapes and performance characteristics, so the generator produces them separately.
- **The `llm_ops_lib` API**: Helper functions in `llm_ops_lib.py` create operators for common patterns. The most important ones are:
  - `create_unary_op()` – Elementwise operations (LayerNorm, RMSNorm, Softmax). Unary operations run on the vector unit (VPU).
  - `create_multi_head_attention()` – Full attention block including Q/K/V projections, attention computation, and output projection. Internally handles GQA via num_kv_heads, and uses FlashAttention for prefill when `use_flash_attention=True`.
  - `create_ffn()` – Feed-forward network. Supports multiple types: `"default"` (2-matrix), `"llama"` (3-matrix SwiGLU), and `"deepseek_moe"` (MoE with expert routing).

[2] https://newsletter.languagemodels.co/p/the-illustrated-gpt-oss

Under the hood: the einsum operator. The high-level functions above (`create_multi_head_attention` and `create_ffn`) are wrappers that internally build up lists of *primitive* operators using lower-level helpers, the most important of which is the einsum. The function `create_einsum_op(input_a_shape, input_b_shape, einsum_expr, ...)` is used to create the matrix-multiplication operators which run on the systolic array (MXU). The shapes of the two inputs and an einsum expression (e.g., "BLM;MND->BLND") fully determine the output shape and FLOP count. The simulator uses these to compute compute-time, memory-time, and the bottleneck.

For example, when `create_multi_head_attention(..., is_decode=False)` is called for prefill, it internally creates several `create_einsum_op` calls for the Q, K, V projections, followed by either a FlashAttention op (if enabled) or explicit $Q{\cdot}K^T$, Softmax, and Attn·V operators, and finally an output projection. For the Q projection specifically, the call looks like:

```
# Inside create_multi_head_self_attention() -- Q projection
create_einsum_op(
    input_a_shape=[batch_size, seqlen, d_model],  # activations
    input_b_shape=[d_model, num_heads, d_head],    # W_q weight matrix
    einsum_expr='BLM;MND->BLND',  # B=batch, L=seqlen, M=d_model
                                  # N=num_heads, D=d_head
    count=count,                  # num_layers (prefill) or
                                  # num_layers*output_seqlen (decode)
    ...
)
# For Llama-3-8B (d_model=4096, num_heads=32, d_head=128):
#   input_a: [1, 4096, 4096], input_b: [4096, 32, 128]
#   output:  [1, 4096, 32, 128]
#   FLOPs = batch * seqlen * d_model * num_heads * d_head * 2
#         = 1 * 4096 * 4096 * 32 * 128 * 2 = ~137 GFLOPs
```

The einsum expression "BLM;MND->BLND" tells the system: input A has axes B, L, M; input B has axes M, N, D; the M axis is contracted (it appears in both inputs but not in the output), and the output has axes B, L, N, D. The FLOP count is computed automatically from these shapes: it equals $2 \times$ batch $\times$ non-reduction dims $\times$ reduction dims. To learn more about einstein summations, you can refer to this resource.[3]

You do not need to call `create_einsum_op` directly for your GPT-OSS implementation—the higher-level `create_multi_head_attention()` and `create_ffn()` handle this for you. But understanding how they decompose into einsum operators will help you reason about the tensor shapes and performance characteristics of the operators your generator produces.

How the simulator evaluates an einsum operator. Once your ops generator produces an einsum operator, the backend (`npusim_lib.py`) determines its execution time by modeling the interaction between compute and memory. The key step is *tiling*: the operation is too large to fit entirely in the on-chip SRAM (Vmem), so the simulator must partition it into tiles that do fit, and estimate how many times data must be loaded from HBM.

Consider a batch matrix multiplication "BMK;BKN->BMN" , where K is the reduction (contracted) axis. The simulator tiles the M and N dimensions into blocks of size m and n:

- **Output tile:** $m \times n$ elements
- **LHS tile:** $m \times k$ elements (a slice of the left input)
- **RHS tile:** $k \times n$ elements (a slice of the right input)
- **Vmem constraint:** All three tiles must fit simultaneously: $m \cdot n + m \cdot k + k \cdot n \leq$ vmem_size

[3] https://rockt.ai/2018/04/30/einsum

The total HBM bytes accessed depends on the tile sizes. With output tiles of size $m \times n$, there are $\frac{MN}{mn}$ output tiles. For each output tile, the LHS slice ($m \times K$) and RHS slice ($K \times n$) must be loaded from HBM. However, tiles that share an m-row can reuse the LHS, and tiles that share an n-column can reuse the RHS. The total traffic works out to the following, assuming E bytes per element:

$$\text{HBM bytes} = E \cdot B \left(MN + \frac{KMN}{n} + \frac{KMN}{m} \right) = E \cdot B \cdot MN \left(1 + K \left(\frac{1}{m} + \frac{1}{n} \right) \right)$$

Larger tiles $\Rightarrow$ smaller $\frac{1}{m} + \frac{1}{n}$ $\Rightarrow$ more data reuse $\Rightarrow$ less HBM traffic. The simulator searches over all valid (m, n, k) factor combinations that satisfy the Vmem constraint, and picks the one that minimizes overall execution time (the max of compute cycles and memory cycles). It also enforces a minimum number of MXU operations per tile (to ensure the systolic array pipeline stays full and can hide HBM latency).

After tiling, the simulator computes:

- **Compute time** = FLOPs / (SA throughput) — determined by the systolic array size and count.
- **Memory time** = HBM bytes accessed / HBM bandwidth — determined by the tile sizes and bandwidth.
- **Execution time** = $\max(\text{compute time}, \text{memory time})$ — the bottleneck determines whether the operator is *compute-bound* or *memory-bound*.

Prefill ops generation. The Llama generator's `generate_prefill_ops()` method builds the operator list for the prefill phase. The core logic (simplified, ignoring parallelism and KV swap boilerplate) is:

```
def generate_prefill_ops(self, fusion_id_start=2):
    ops = []
    fusion_id = fusion_id_start

    # 1) LayerNorm before attention (all layers)
    ops.append(ops_lib.create_unary_op(
        input_shape=[self.batch_size, self.input_seqlen, self.d_model],
        op_name=LayerNorm,
        count=self.num_layers,   # repeated for every layer
        fusion_id=fusion_id, ...
    ))
    fusion_id += 1

    # 2) Multi-head attention (all layers)
    ops += ops_lib.create_multi_head_attention(
        batch_size=self.batch_size,
        input_seqlen=self.input_seqlen,
        output_seqlen=self.output_seqlen,
        num_heads=self.num_heads,
        d_model=self.d_model,
        d_head=self.d_head,
        num_layers=self.num_layers,
        is_decode=False,                # prefill
        use_flash_attention=True,       # fused Q*K*V kernel
        num_kv_heads=self.num_kv_heads,  # for GQA
        fusion_id_start=fusion_id, ...
    )
```

```
    fusion_id = ops[-1].fusion_id + 1

    # 3) FFN (all layers)
    ops += ops_lib.create_ffn(
        batch_size=self.batch_size,
        input_seqlen=self.input_seqlen,
        d_model=self.d_model,
        d_ff=self.d_ff,
        num_layers=self.num_layers,
        ffn_type=self.ffn_type,  # llama = SwiGLU (gate/up/down)
        is_decode=False,
        fusion_id_start=fusion_id, ...
    )
    return ops
```

There are a few important things to note about this code:

- **The count field**: Each operator represents a single layer's computation, but `count=self.num_layers` tells the simulator this operator is repeated 32 times (for Llama-3-8B's 32 layers). The simulator multiplies the per-invocation execution time by the count. This is more efficient than creating 32 separate operator objects with identical shapes.
- **Fusion IDs**: Every operator has a `fusion_id` that groups related ops. After adding ops, always update `fusion_id`, so subsequent ops get unique IDs.
- **Prefill tensor shapes**: During prefill, the sequence dimension is `input_seqlen` (e.g., 4096). The Q/K/V projections operate on `[batch_size, input_seqlen, d_model]`, producing large matrices that make prefill compute-bound.

Decode ops generation. The decode phase is structurally similar but with different shapes and counts:

```
def generate_decode_ops(self, fusion_id_start=2):
    ops = []
    fusion_id = fusion_id_start
    count = self.num_layers * self.output_seqlen  # layers x tokens

    # 1) LayerNorm (seqlen=1, one token at a time)
    ops.append(ops_lib.create_unary_op(
        input_shape=[self.batch_size, self.decode_width, self.d_model],
        op_name=LayerNorm,
        count=count,   # num_layers * output_seqlen
        fusion_id=fusion_id, ...
    ))
    fusion_id += 1

    # 2) Attention (reads from full KV cache)
    ops += ops_lib.create_multi_head_attention(
        batch_size=self.batch_size,
        input_seqlen=self.input_seqlen,   # KV cache from prefill
        output_seqlen=self.output_seqlen, # KV cache from decode
        decode_width=self.decode_width,   # typically 1
        num_layers=self.num_layers,
        is_decode=True,                   # decode mode
```

```
        num_kv_heads=self.num_kv_heads,
        fusion_id_start=fusion_id, ...
    )
    fusion_id = ops[-1].fusion_id + 1

    # 3) FFN
    ops += ops_lib.create_ffn(
        ..., is_decode=True, ...
    )
    return ops
```

The key differences from prefill:

- **count = num_layers × output_seqlen**: Decode generates tokens one at a time, so we repeat the full layer stack for each output token.
- **decode_width = 1**: Each decode step processes a single token, so the "sequence length" dimension in the input is 1 instead of `input_seqlen`.
- **KV cache**: In decode, `create_multi_head_attention` uses `input_seqlen` and `output_seqlen` to determine the KV cache size. The attention matmul is $Q_{[1]} \times K_{[\text{input\_seqlen}+\text{output\_seqlen}]}$, which is a tiny compute operation but requires loading the entire KV cache from HBM—making decode memory-bandwidth-bound.

Memory footprint. The `compute_memory_footprint_bytes()` method estimates how much HBM is needed. For Llama, it delegates to `memory_footprint_analysis_lib.get_llm_inference_mem_requirement()`, which sums up model weights (attention projections + FFN matrices) and the KV cache. This is separate from memory *traffic* (how much data is read/written during execution)—footprint is the static capacity needed to hold the model and its state.

What to keep in mind for GPT-OSS. When implementing the GPT-OSS ops generator, the key differences from Llama are:

1. **Two attention types**: GPT-OSS alternates between full attention and sliding window attention layers. Instead of having a single `create_multi_head_attention` call with `count=num_layers`, you'll need separate calls for full layers and sliding window layers, each with their own count.
2. **MoE FFN**: Use `ffn_type="deepseek_moe"` instead of `"llama"`. The config fields (`num_routed_experts`, `moe_d_ff`, etc.) are read automatically.
3. **Mixed KV cache sizes**: For memory footprint, full-attention layers have KV cache scaling with sequence length, while sliding window layers are bounded at `sliding_window_size`. You need to account for both.

2.2.5 Integrating the Ops Generator with the Simulator

Now you're ready to test your model. The provided patch includes the scaffolding code necessary to add your model as an option when launching the simulator. Specifically, it imports the new model in `neusim/run_scripts/run_sim.py`, and adds "gpt-oss" as an option in the `generate()`, `dump_stats()`, `dump_stats_llm()`, and `map_parallelism_to_ici_axes()` functions.

Deliverables for § 2.2: After extending the simulator to support the GPT-OSS model, provide the following:

1. Create a patch file called `gpt-oss-part-2.patch` that contains all the code changes you made for this part.

2. Compare the computational cost (FLOPs) and memory traffic of full attention with sliding window attention operators and plot the results using input sequence lengths of 512, 1024, 2048, 4096, and 8192, with a sliding window size of 128. You may find `neusim/npusim/frontend/run_single_op.py` helpful. Then, explain the benefits and drawbacks of using sliding window attention, citing the simulation results where applicable.

3. Run the simulator on the GPT-OSS-20b model for input sequence lengths of 1024, 2048, 4096, and 8192 for both prefill and decode. Use an output sequence length of 128. Plot the total memory footprint, memory traffic, and compute (FLOPs) against the sequence length.

4. Draw a computational graph for the operators of two layers produced by the `OpsGenerator` you implemented. Do this for both prefill and decode. The graph should contain the tensor dimensions for each operator (inputs, weights, outputs). You should draw this with a computer. We recommend using draw.io, but you may use any tool you'd like.

2.3 Sensitivity Analysis

In this part of the MP, you will change the configurations of chip components inside an NPU to examine the impact of these components on end-to-end inference performance. Use the following settings:

- Model: GPT-OSS-20b
- Baseline chip configuration: TPUv5p (`configs/chips/tpuv5p.json`)
- Number of chips: 1
- Batch size: 4
- Input Sequence Length: 4096
- Output Sequence Length: 128

How to vary parameters. For your convenience, the patch file provided in §2.2 includes a script that sweeps a list of values for a single parameter at a time. You can find the script at `neusim/run_scripts/sweep_chip_params.py`. Use it to run a sensitivity analysis for the sweep ranges listed below. The script will write its results to `results/sweep/sweep_summary.csv`. With 5 parameter values $\times$ 5 different parameters, you will run 25 total simulations. The simulations should complete within a few seconds.

```
python neusim/run_scripts/sweep_chip_params.py \
    --model configs/models/gpt-oss-20b.json \
    --batch_sizes 4 --input_seqlen 4096 --output_seqlen 128
```

By default, the script will sweep the following parameters:

1. Number of systolic arrays (`num_sa`): 1, 2, 4, 8, 16
2. Systolic array size (`sa_dim`): 32, 64, 128, 256, 512
3. Number of vector units (`num_vu`): 1, 2, 4, 6, 12 (also set `num_vu_ports` to the same value)
4. Main memory (HBM) bandwidth (`hbm_bw_GBps`): 500, 1000, 2000, 2765, 4000
5. Vector memory (SRAM scratchpad) size (`vmem_size_MB`): 16, 32, 64, 128, 256

Deliverables for § 2.3: After completing the simulations, please analyze the results as follows:

1. Based on the results you obtained, plot the sensitivity of prefill and decode latency to changes in each of the parameters above. For each hardware parameter, you should plot the following two relationships: prefill latency (time to first token, TTFT) vs. the parameter value, and decode (time per output token, TPOT) latency vs. the parameter value.

2. Using the plots you just made, examine the impact to end-to-end LLM inference performance when changing the value of each hardware parameter. Comment on any trends you observe. If you observe no changes in performance when scaling a resource, explain why.

3. Which operators are processed by the vector unit? Which operators are processed by the systolic array? Explain the strengths and weaknesses of each unit.

4. What is the difference between scaling the number of systolic arrays versus the systolic array dimension? Give an example of a situation in which you would choose to do each.

3 Understanding the Performance Bottlenecks of LLM inference

In this section, we dive deeper into the LLM inference process to understand the resource demands of each stage. You will examine the individual operators produced by the simulator for both prefill and decode, compute their arithmetic intensity, and classify them as compute-bound or memory-bound. Following this, you will study how request characteristics (batch size) shift these bottlenecks, and how model parallelism strategies (tensor parallelism and pipeline parallelism) affect performance at scale.

3.1 Understanding Operator-Level Bottlenecks

The simulator produces per-operator CSV traces for both prefill and decode. Each row in the CSV contains, among other fields:

- `Execution time (ns)`: The total time for this operator (the bottleneck of compute vs. memory).
- `Compute time (ns)`: Time if the operator were purely compute-limited.
- `Memory time (ns)`: Time if the operator were purely memory-bandwidth-limited.
- `FLOP count`: Number of floating-point operations.
- `Bytes accessed`: Total bytes read from and written to HBM.
- `Count`: How many times this operator is repeated (e.g., once per layer, or once per layer per output token).

An operator is **compute-bound** when its compute time exceeds its memory time (i.e., the systolic array or vector unit is the bottleneck), and **memory-bound** when its memory time exceeds its compute time (i.e., HBM bandwidth is the bottleneck). Whether an operator is compute- or memory-bound depends on its *arithmetic intensity*: the ratio of FLOPs to bytes accessed. The TPUv5p has a peak compute throughput of approximately 459 TFLOPS (BF16) for its systolic array and an HBM bandwidth of 2765 GB/s, so saturating its compute throughput requires a minimum arithmetic intensity of:

$$\frac{\text{Peak FLOPS}}{\text{HBM BW}} = \frac{459 \times 10^{12}}{2765 \times 10^{9}} \approx 166 \text{ FLOPs/byte}$$

Operators with arithmetic intensity above this threshold are compute-bound; those below are memory-bound.

3.1.1 LLM Prefill

Open the prefill CSV file from your sensitivity analysis baseline run (§ 2.3). Each operator in the CSV has a human-readable `Description` column and a more detailed `Name` column—you can identify operators using substring matches in either column. For each of the following operator categories, compute the arithmetic intensity (FLOPs ÷ bytes accessed) from the CSV data, classify the operator as compute-bound or memory-bound, and answer any additional questions for that operator. Show your work.

1. **LayerNorm** (`Description` contains "Input_layernorm" or "Attention_layernorm"). This is an elementwise operation that runs on the vector unit (VPU). What are its input dimensions? How many FLOPs does it perform? How many bytes does it access?
2. **Q projection** (`Description` is "-Q-3"). This is an einsum/matrix multiplication that runs on the systolic array (MXU). What are the dimensions of the input activation and weight tensors?
3. **Attention score computation** (`Description` contains "FlashAttention"). Because FlashAttention fuses $Q \cdot K^\top$, softmax, and Attn·V into one kernel, this single operator represents the entire attention computation.

4. **FFN gate and up projections** (`Description` is "Fwd-FFN-encoder-FFgate" and "Fwd-FFN-encoder-FFup"). What is the intermediate dimension `d_ff`?

5. **FFN activation** (`Description` is "Fwd-FFN-encoder-FFgate_up"). This is the elementwise SwiGLU activation that runs on the VPU.

6. **FFN down projection** (`Description` is "Fwd-FFN-encoder-FFoutput").

Deliverables for 3.1.1:

1. For each operator category above, report the FLOP count, bytes accessed, arithmetic intensity (FLOPs/byte), and whether it is compute-bound or memory-bound. Present this as a table.

2. Based on your analysis, would you say that the LLM prefill stage is compute-bound or memory-bound? Justify your answer by considering which operators dominate the total execution time.

3. How would your answer change if the input sequence length were much shorter (e.g., 128 tokens)? Which operators would shift from compute-bound to memory-bound, and why? (*Hint: think about how the arithmetic intensity of the Q/K/V projections changes when the batch of tokens being processed shrinks.*)

3.1.2 LLM Decode

Now open the decode CSV file. Repeat the same analysis for the decode phase. The decode stage uses the KV cache built up during prefill, so its attention operator structure is different: instead of one FlashAttention kernel, you will see separate matmuls for the Q, KV projections and for the Q·K and attention-weight·V products. Note also that the KV cache is split into a "prefix" (the tokens from prefill, of length `input_seqlen`) and a "suffix" (the tokens generated so far during decode, up to `output_seqlen`).

For each of the following operator categories, compute the arithmetic intensity and classify as compute-bound or memory-bound.

1. **LayerNorm** (`Description` contains "Input_layernorm" or "Attention_layernorm").

2. **Q projection** (`Description` is "Attention-serving-decode-Q/K/V" and the `Name` column contains "MatMulQ").

3. **KV projection** (`Description` is "Attention-serving-decode-Q/K/V" and the `Name` column contains "MatMulKV").

4. **Attention prefix Q·K** (`Description` is "Attention-serving-decode-Softmax(Q*K)*V" and the `Name` column contains "QKprefix"). This attends to the KV cache built up during prefill (length `input_seqlen`).

5. **Attention suffix Q·K** (`Description` is "Attention-serving-decode-Softmax(Q*K)*V" and the `Name` column contains "QKsuffix"). This attends to the KV cache accumulated during decode so far (length up to `output_seqlen`).

6. **Softmax** (`Description` is "Attention-serving-decode-Softmax(Q*K)*V" and runs on the VPU—look for the row with `OpType=VPU` that contains "Softmax" in the `Name`).

7. **Output projection** (`Description` is "Attention-serving-decode-Attention_output").

8. **FFN gate and up projections** (`Description` is "Fwd-FFN-serving-decoder-FFgate" and "Fwd-FFN-serving-decoder-FFup").

9. **FFN activation** (`Description` is "Fwd-FFN-serving-decoder-FFgate_up").

10. **FFN down projection** (`Description` is "Fwd-FFN-serving-decoder-FFoutput").

Note that in decode, the count field for each operator equals num_layers × output_seqlen, because each operator is invoked once per layer for every output token generated. When computing arithmetic intensity, use the per-invocation FLOPs and bytes.

Deliverables for 3.1.2:

1. For each operator category above, report the same table as for prefill: FLOP count, bytes accessed, arithmetic intensity, and bound classification.
2. Based on this analysis, is the decode stage primarily compute-bound or memory-bound? Justify your answer.
3. Compare the arithmetic intensity of the Q projection in prefill vs. decode. The weight matrix W_Q is identical in both phases. Why is the arithmetic intensity so different? (*Hint: in prefill, the "batch" of tokens being projected has size* `input_seqlen`*; in decode, it has size 1.*)
4. The attention score operators (the prefix and suffix Q·K matmuls) in decode access the full KV cache from HBM. How do the compute cost (FLOPs) and memory-access cost (bytes) of these operators scale with the input sequence length? Is this cost significant compared to the other operators in the decode stage? Does this make the attention operators more or less memory-bound as sequence length grows?

3.1.3 Understanding the Impact of Batch Size

The performance profile of LLM inference depends heavily on the characteristics of the input request. In this section, you will study how the *batch size* affects the execution time, bottleneck classification, and resource utilization of both prefill and decode.

Experiments. Use the GPT-OSS-20B model (`configs/models/gpt-oss-20b.json`) with the TPUv5p baseline configuration on a **single chip** (no tensor, pipeline, or data parallelism). Fix the input sequence length to 2048 and the output sequence length to 128, and sweep over the following batch sizes: **1, 4, 16, 64**. You can use the provided `sweep_chip_params.py` script with `--batch_sizes 1,4,16,64 --params num_sa --model configs/models/gpt-oss-20b.json` (and ignore the `num_sa` dimension in your analysis—you only need the baseline chip config, not the sweep).

Deliverables for 3.1.3:

1. Plot the prefill execution time and the decode execution time (separate lines, or a grouped bar chart) against the batch size.
2. How does batch size affect the prefill throughput and latency (TTFT)? Explain why based on simulation results and your conceptual understanding. (*Hint: Think about arithmetic intensity.*)
3. How does batch size affect the *decode* throughput and latency (TPOT)? Does the per-token decode time stay constant, decrease, or increase as the batch size grows? Explain why.
4. For decode, examine the per-operator CSV traces for the smallest (batch size 1) and largest (batch size 64) batch sizes. For each, identify which operators are compute-bound vs. memory-bound. Do any operators change their bottleneck classification as the batch size increases? Which ones? Does this shift the bottleneck for the decode stage as a whole?
5. Based on your answers to the previous questions, does prefill benefit significantly from batching? How about decode? Consider resource utilization, latency, and throughput when explaining your answer. Do the best batch sizes for prefill and decode differ?

3.2 Examining Parallelism Strategies for Large Models

The GPT-OSS-20B model fits on a single chip, but larger models such as GPT-OSS-120B, DeepSeek-v2-236B, and Llama-3.1-405B need to be partitioned across multiple chips due to their much larger memory footprints. We will use NeuSim to investigate two forms of model parallelism:

- **Tensor Parallelism (TP)**: Splits each operator's weight matrices across chips. Each chip computes a shard of every layer, then the chips communicate (via AllGather/ReduceScatter over the inter-chip interconnect, ICI) to reconstruct the full result. TP reduces per-chip compute and memory, but adds communication after every layer.
- **Pipeline Parallelism (PP)**: Splits the model into pipeline stages, where each stage consists of several layers. Each stage is assigned to a subset of chips. The chip(s) assigned to a given pipeline stage run all operators for the layers for that stage, then send the activations to the next stage in the pipeline. PP adds communication only between pipeline stages (once per stage).

A note on pipeline parallelism in NeuSim. NeuSim models pipeline parallelism by simulating only *one pipeline stage on one chip for one microbatch*, then extrapolating end-to-end metrics from that single-stage measurement. Specifically:

- **Layer sharding.** Each chip's ops generator is only asked to produce ops for $\lceil \text{num\_layers}/\text{PP} \rceil$ layers (its share). All subsequent per-operator timing reflects this reduced layer count.
- **Microbatch size.** NeuSim determines the per-stage tensor batch dimension as $\text{microbatch\_size} = \lceil \text{global\_batch\_size}/\text{PP} \rceil$ (assuming no data parallelism). This is the batch dimension fed to the ops generator; it implicitly assumes the global batch is split into exactly PP microbatches, one per pipeline stage. Any ceiling rounding when $\text{global\_batch\_size} < \text{PP}$ leaves the pipeline underfilled.
- **End-to-end latency.** TTFT and TPOT are computed as per-stage time$\times$PP, modeling a single request serially traversing every stage.
- **Throughput.** The `throughput_tokens_per_sec` field in the output JSON is computed as microbatch_size$\times$seqlen/per-stage time. **This is a steady-state upper bound that assumes the pipeline is always full—i.e., it assumes PP microbatches are in flight at once.** If the global batch size is smaller than the pipeline depth, this number is optimistic: the actual single-request throughput is global_batch_size $\times$ seqlen/TTFT, which can be up to PP$\times$ smaller. Pipeline bubbles (the fill/drain overhead) are not modeled.

At `batch_size=1` with $\text{PP} = 16$, NeuSim's reported throughput assumes 16 microbatches in flight but there is actually only one microbatch (the single request), so the reported number overstates the achievable throughput by roughly a factor of 16. At `batch_size=64` with $\text{PP} = 16$, the global batch can be split into 16 microbatches (one per pipeline stage), so the pipeline can be filled and the reported throughput is approximately accurate (ignoring fill/drain bubbles). This section is just for your reference; you may directly report the extrapolated results in your experiments, but understanding how the simulator works may be helpful when explaining the numbers. The provided script will handle setting the microbatch sizes.

How to run multi-chip simulations. We provide a script to help you run the experiments.[4] The key parameters to set are `num_chips`, `tensor_parallelism_degree`, `pipeline_parallelism_degree`, and the corresponding `num_*_parallel_axes` fields. The axis fields control how many physical ICI mesh dimensions each parallelism strategy occupies on the TPUv5p's 3-D torus (so the total across all parallelism types must not exceed 3). You may find it helpful to reference the functions `split_parallelism_degree` and `run_sim_lib.map_parallelism_to_ici_axes`:

[4] https://raw.githubusercontent.com/MichaelWang237/ece511-2026-public/refs/heads/main/mp4/sweep_parallelism.py

- When only TP is enabled (PP $= 1$, TP > 1): TP is allowed to use all 3 axes. The `split_parallelism_degree` function ensures that determines how many axes may be utilized after accounting for the 3D torus topology.
- When both TP and PP are enabled: PP takes 1 axis (it only needs one for its point-to-point communication), and TP takes the remaining 2 axes.
- When only PP is enabled (TP $= 1$, PP > 1): PP uses 1 axis, TP uses 0.
- When a parallelism degree equals 1, its axis count should be 0.

Experiments. For this analysis, we will use the following three large models with the TPUv5p chip configuration and a total of 16 chips:

- GPT-OSS-120B: `configs/models/gpt-oss-120b.json` (MoE with sliding window attention; use your `GptOssOpsGenerator`)
- DeepSeek-v2-236B: `configs/models/deepseekv2-236b.json` (MoE with MLA; use `DeepSeekOpsGenerator`)
- Llama-3.1-405B: `configs/models/llama3_1-405b.json` (dense; use `LLMOpsGeneratorInference`)

Use the following settings (these are provided in the script):

- Microbatch sizes: 1, 4, 16, 64
- Input sequence length 2048, output sequence length 128
- (TP, PP) configurations ensuring TP $\times$ PP $= 16$:

TP	PP	Description
1	16	Pure pipeline parallelism
2	8	Mostly pipeline
4	4	Balanced
8	2	Mostly tensor
16	1	Pure tensor parallelism

You should have 3 models $\times$ 4 microbatch sizes $\times$ 5 parallelism configurations, for a total of 60 simulation runs.

Deliverables for 3.2:

1. Plot the prefill and decode latency (TTFT, TPOT) (y-axis) against the tensor parallelism degree (x-axis) for all five configurations. Create a separate plot for decode and prefill for each of the models (6 plots total). In each plot, there should be four lines that show the relationship between latency and parallelism configuration for each of the four microbatch sizes.
2. Plot the prefill and decode throughput (tokens per second) for the different configurations. Follow the same structure outlined in the previous deliverable.
3. Plot the total ICI communication volume (bytes) for prefill and decode against the tensor parallelism degree. Use the same layout as the latency plot above: one subplot per model and phase (6 plots total), with one line per microbatch size.
4. For the **prefill** stage: How does increasing TP change performance (latency, throughput)? Use the plots you just generated, and explain any trends you observe. Do these trends vary across different batch sizes and models (dense vs. MoE)? Explain your answer from a conceptual standpoint using your understanding of the operators shapes/characteristics in the prefill stage.

5. For the **decode** stage: How does increasing TP change performance (latency, throughput)? Use the plots you just generated, and explain any trends you observe. Do these trends vary across different batch sizes and models (dense vs. MoE)? Explain your answer from a conceptual standpoint using your understanding of the operators shapes/characteristics in the decode stage.

6. Compare the parallelism configuration which delivers the best throughput for prefill and the parallelism configuration that delivers the best throughput for decode. Are there any trends you observe? Do different microbatch sizes change this trend? Explain why using your understanding of the resource demands of each inference stage.

7. Answer the above question, but this time using latency as the objective (TTFT/TPOT). How do the ideal parallelism configurations compare for prefill and decode now? Explain your answer.

8. Based on your investigation of parallelism strategies for prefill and decode, would it be beneficial to use different parallelism strategies for each phase? Justify your answer with specific references to the TTFT, TPOT, and throughput trends you observed.

4 PD Disaggregation

In Part III, you found that the prefill and decode stages respond very differently to batch size and parallelism configuration: prefill is compute-intensive and benefits from higher tensor parallelism, while decode is largely memory-bandwidth-bound and benefits from the increased reuse opportunities provided by larger batch sizes. This asymmetry has led providers to explore *Prefill-Decode (PD) disaggregation* [6]; instead of running both phases on the same nodes, forcing prefill and decode to compromise on the parallelism and batch size decisions, we can **physically decouple** the two phases and configure the nodes used for each optimally. That is, a dedicated set of *prefill nodes* handles prompt processing, and a separate set of *decode nodes* handles token generation. After a prefill node finishes processing an input prompt, it transfers the resulting KV cache to a decode node over the data center network (DCN). The overall system throughput depends on both pools and the cost of that KV transfer.

In this part, you will implement PD disaggregated serving in NeuSim and estimate the achievable system throughput.

4.1 Implementing Disaggregated Serving in NeuSim

`LLMOpsGenerator` already exposes `generate_prefill_ops()` and `generate_decode_ops()` as separate methods. The current `generate()` method calls both from a *single* config, because NeuSim assumes co-located serving. To support PD disaggregation, you need to instantiate two independent config objects—one for the prefill pool and one for the decode pool—and simulate each phase under its own parallelism and batch-size configuration.

4.1.1 Task 1: Separate Prefill and Decode Simulation

Create a new script `run_pd_disagg.py`. The script will need to accept the following parameters (at minimum):

- Model config path (e.g., `configs/models/gpt-oss-20b.json`)
- Chip config path (e.g., `configs/chips/tpuv5p.json`)
- Prefill pool: `--prefill_tp`, `--prefill_pp`, `--prefill_batch_size`
- Decode pool: `--decode_tp`, `--decode_pp`, `--decode_batch_size`
- `--total_chips` (total chip budget, to be allocated between prefill and decode)
- `--output_dir`

The script should perform the following steps:

1. Build a **prefill config**: use the parameters specified by the model JSON, chip JSON, and the prefill pool's TP/PP/batch-size parameters. Set `num_chips` to the number of chips *per prefill instance* (i.e., `prefill_tp * prefill_pp`).
2. Build a **decode config**: same model and chip JSON, but with the decode pool's TP/PP/batch-size. Set `num_chips` to the number of chips *per prefill instance* (i.e., `decode_tp * decode_pp`).
3. Instantiate an ops generator with the prefill config, call `generate_prefill_ops()`, and then call `analysis_lib.fill_operators_execution_info()` on the resulting ops list. Dump to a CSV file and call `run_sim_lib.get_statistics_from_trace_file()` to obtain timing stats. Do the same for the decode config using `generate_decode_ops()`.
4. Compute TTFT, TPOT, prefill throughput, and decode throughput using the same logic as `run_sim.py:dump_stats_llm()`.
5. Compute KV cache transfer time (see § 4.1.2) and add it to TTFT.

6. For now, allocate 1/2 of the `--total_chips` to prefill and 1/2 to decode. Using this, compute the number of possible prefill and decode instances, and use this information to derive the system-level throughput (i.e., how many queries can be processed by all instances).

7. Write a JSON summary containing: TTFT (with and without KV transfer), TPOT, per-instance prefill throughput, per-instance decode throughput, number of prefill instances, number of decode instances, and system throughput.

Note: You should not need to modify any existing NeuSim source files. Both `generate_prefill_ops()` and `generate_decode_ops()` are already available; you are only writing a new orchestration script.

4.1.2 Task 2: KV Cache Transfer Overhead

After prefill completes, the KV cache must be transferred from the prefill node to the decode node. This transfer may consist of large amounts of data as input sequence lengths increase, and thus the transfer time may become significant. You will model it in your PD disaggregated serving system.

KV cache size. In § 2.2, you implemented `compute_memory_footprint_bytes()` for `GptOssOpsGenerator`, which internally uses `memory_footprint_analysis_lib.get_llm_inference_kv_cache_mem_requirement(config)` to compute the KV cache size. Reuse this function here. The transfer must move the *full* (unsharded) KV cache. Because `get_llm_inference_kv_cache_mem_requirement` divides by the config's tensor-parallelism degree, pass the config with `tensor_parallelism_degree=1` (and `tensor_parallel_degree_dcn=1`) when computing the transfer size, even if the prefill instance uses TP.

Transfer time.

$$t_{\text{KV}} = \frac{\text{KV\_size\_bytes}}{B_{\text{DCN}}}$$

where B_{DCN} is the DCN bandwidth in bytes/s. Assume a non-pipelined transfer: the full KV cache is sent after prefill computation completes, and decode cannot start until the transfer finishes. Add t_{KV} to TTFT:

$$\text{TTFT}_{\text{total}} = \text{TTFT}_{\text{prefill}} + t_{\text{KV}}$$

Add a function `compute_kv_cache_transfer_time_ms(config, dcn_bw_GBps)` to your script.

Validation. Before running experiments, verify your implementation: Run `run_pd_disagg.py` with *identical* prefill and decode configs (same TP, PP, batch size as the TPUv5p baseline). Confirm that your TTFT and TPOT values match `sweep_chip_params.py` for the same model and batch size (excluding the KV transfer term). Also compare your computed KV cache transfer time to your interconnect bandwidth and computed KV cache size. Ensure these values are consistent.

Deliverables for 4.1:

- Submit a patch file containing `run_pd_disagg.py` and any other modified files.
- Include the validation results in your report.

4.2 Chip Allocation Sweep

So far, you are allocating an equal number of chips to both prefill and decode. However, this does not necessarily improve system throughput, as prefill and decode will not always need the same number of resources.

4.2.1 System Throughput Model

With N_P chips allocated to the prefill pool and N_D chips to the decode pool ($N_P + N_D = N_{\text{total}}$), and each prefill instance using $C_P = \text{TP}_P \times \text{PP}_P$ chips and each decode instance using $C_D = \text{TP}_D \times \text{PP}_D$ chips, the pool sizes are:

$$n_P = \lfloor N_P / C_P \rfloor, \quad n_D = \lfloor N_D / C_D \rfloor$$

For a valid allocation, N_P must be a multiple of C_P and $N_D = N_{\text{total}} - N_P$ must be a multiple of C_D.
The per-instance *request throughput* (requests served per second) for each pool is:

$$\lambda_P = \frac{b_P}{T_{\text{prefill}}}, \quad \lambda_D = \frac{b_D}{T_{\text{decode}}}$$

where b_P and b_D are the batch sizes, T_{prefill} is the prefill wall-clock time in seconds (TTFT excluding KV transfer), and $T_{\text{decode}} = \text{TPOT\_ms\_request} \times S_{\text{out}}/1000$ is the total decode time in seconds for one batch (`TPOT_ms_request` is the per-token latency reported by NeuSim, so multiply by output sequence length to get the full decode time).
The **system request throughput** (requests per second) is limited by the slower pool:

$$\Lambda_{\text{sys}} = \min(n_P \cdot \lambda_P,\ n_D \cdot \lambda_D)$$

When $n_P \lambda_P = n_D \lambda_D$, the system is balanced and Λ_{sys} is maximized. In this task, you will tune the pool sizes to reach this maximal throughput.

4.2.2 Task 3: Allocation Sweep for GPT-OSS-120B

Use **GPT-OSS-120B** (`configs/models/gpt-oss-120b.json`, `GptOssOpsGenerator`) with the **TPUv5p** chip, `input_seqlen=2048`, `output_seqlen=128`, and a total budget of $N_{\text{total}} = \mathbf{256}$ chips.

Step 1: Choose pool configurations. From your Part III results (§ 3.2), select:

- The (TP, PP, batch size) configuration that maximized *prefill throughput* on 16 chips.
- The (TP, PP, batch size) configuration that maximized *decode throughput* on 16 chips.

Step 2: Enumerate and sweep valid allocations. With $N_{\text{total}} = 256$, N_P must be a multiple of C_P and $N_D = 256 - N_P$ must be a multiple of C_D. List all valid allocations and run `run_pd_disagg.py` for each. Report the performance.

Deliverables for 4.2.2:

1. Report the chosen (TP, PP, batch size) for each pool.
2. List all valid allocations and plot the prefill pool capacity $n_P \lambda_P$, the decode pool capacity $n_D \lambda_D$, and the system throughput Λ_{sys} (all in requests/s) as a function of N_P. Mark the N_P value which produces the best Λ_{sys}. Include this plot in the report.
3. What is the optimal N_P that maximizes system throughput? How does it compare to an even 50/50 split? Is the system easy or difficult to balance given the constraint that N_P must be a multiple of C_P? Explain intuitively why the optimal allocation is where it is.
4. How does the KV transfer overhead (t_{KV}) compare to the prefill compute time? Plot this overhead as you vary the interconnect bandwidth. What fraction of TTFT does it represent at DCN bandwidths of 200 GB/s and 50 GB/s?

5. How does Λ_{sys} at the optimal allocation compare to a baseline where all 256 chips run co-located serving (no disaggregation) with the same batch size? Quantify the improvement (or lack thereof) and explain.

4.3 Discussion

Deliverables for 4.3:

1. **Throughput vs. SLO tradeoff.** In a real serving system, operators care about *goodput*: the fraction of requests that satisfy latency SLOs (e.g., TTFT $\leq 2\,\text{s}$, TPOT $\leq 50\,\text{ms/token}$). Does optimizing for system throughput Λ_{sys} necessarily yield the best SLO attainment? Under what conditions might they conflict?

2. **Dynamic allocation.** The optimal N_P/N_D split depends on the workload mix. If the average input sequence length increases from 2048 to 8192 tokens (while output length stays at 128), how would you expect the optimal allocation to shift? Why?

3. **Limitations.** Our model makes several simplifying assumptions. Identify at least **two** that could significantly affect results in a real system, and briefly explain how each would change the analysis.

5 Improve the NPU Architecture for PD Disaggregation

In this section, your objective is to design an improved architecture for PD disaggregated inference on the NPU. This portion of the MP is open-ended. We expect you to think critically about system-level bottlenecks before proposing and implementing your solution. While optimizing the provided hardware configurations is a valid approach, we also encourage you to go further and try modifying the simulator codebase to introduce novel components, custom memory hierarchies, or any other big ideas you can think of.

5.1 Optimization Task

Your goal is to maximize the achieved throughput of your `gpt-oss-120b` model on a NPU inference system under the following set of constraints:

- Latency (SLOs). You are given a maximum TTFT of 2000 ms and TBT SLO of 100 ms. Your design must meet these latency constraints for a homogenous workload with `input_seq_len = 2048` and `output_seq_len = 128`.
- Physical constraints. We provide an area model for on-chip resources and a pin model for off-chip bandwidth. You will use these to set the parameters of your NPU chips. The maximum size of a single chip is 858 mm^2 (as constrained by the reticle limit).
- Cost. You are given a $450,000 budget to "buy" logic (die area), HBM modules (derived from memory bandwidth), and ICI links (inter-chip communication bandwidth). Your overall serving system (i.e., the number and size of chips in your cluster) is constrained by this budget.

You should use the area, power, and cost models provided (see § 5.2) to ensure your final design follows these constraints. If you choose to make significant modifications (e.g., add new components) to the NPU device, you will need to modify the provided models to account for your changes when checking your architecture against the constraints.

Deliverables for 5.1:

- **Code Submission:** Provide a patch file containing all modifications made to the baseline simulator codebase.
- **Performance Evaluation:** Report the achieved overall system throughput alongside the phase-specific prefill and decode throughputs and latencies. Include any empirical data visualizations (e.g., parameter sweep graphs) utilized to inform and justify your architectural decisions.
- **Architecture and Methodology:** Detail your final architectural modifications, including a complete specification of your selected parameters. Provide a justification for your design choices by outlining the methodology used to evaluate potential solutions. Focus on explaining why your proposed design will improve upon the configurations explored in previous parts of this MP.
- **Constraint Analysis and Trade-offs:** Analyze the primary architectural trade-offs required to satisfy the provided area, perimeter, and cost models. Identify the most severe limiting factor in your optimization process: was your design predominantly constrained by silicon area (A_{total}), physical I/O edge perimeter, or the overall cluster budget (C_{total})?
- **Objective Function Shift:** Discuss how your architectural parameters and PD disaggregation strategy would adapt if the primary objective function were shifted from maximizing throughput to minimizing end-to-end latency. Contrast this hypothetical latency-optimized architecture with your proposed throughput-optimized design.

- **Model Fidelity and Physical Realities:** The constraint models detailed in § 5.2 are rough approximations to help determine feasibility. While they account for the relative area contributions of the core datapath and memory components (e.g., Systolic Arrays, Vector Units, HBM, and ICI), they abstract away details of the frontend and control logic. These models also do not consider physical design challenges such as yield, thermal dissipation limits (power density), and package-level routing congestion. Analyze how these omissions could impact your design and evaluation. Discuss the practical engineering challenges that would arise when translating your simulated parameters into a physical NPU deployment. Specifically, how might the hidden complexities make your proposed architecture more difficult to realize than the simulator suggests?
- **Workload Generalization and Heterogeneity:** This exercise requires optimizing for a single, homogeneous workload (i.e., fixed input and output sequence lengths). Would your optimized architecture generalize to variable workloads? Why or why not?

5.2 Ensuring Design Feasibility

To help you validate if your design is feasible or not, we provide an area, perimeter, and cost model, in the form of a python file (`feasibility.py`) available to download at the link below[5]. This will help you check that your chip components fit within a standard size die and compute the area they occupy, as well as ensure that you have sufficient perimeter to support the desired total bandwidth of communication (i.e., with the HBM and via the ICI).

Area Model

The silicon area model is calibrated against a TSMC N5 (5nm) process node, which is representative of modern accelerator designs like the TPU v5p/v6p. The total silicon area of your design is computed as:

$$A_{total} = (A_{SA} + A_{VU} + A_{VMEM} + A_{ICI} + A_{HBM}) \times r_{routing} + A_{ctrl} + A_{unmodeled}$$

where $r_{routing}$ is a routing overhead multiplier that accounts for power delivery network (PDN) stripes, clock distribution, and signal routing whitespace. A_{ctrl} and $A_{unmodeled}$ are fixed additive constants explained below.

The scalable components each grow with your design choices:

- **Systolic Arrays (A_{SA}):** Scales with the total number of MAC cells across all arrays, i.e., $N_{SA} \cdot D_{SA}^2$. Covers compute cells, local column FIFOs, and activation pipeline registers.
- **Vector Units (A_{VU}):** Scales with the total number of SIMD lanes across all VUs. Each VU has $8 \times 128 = 1024$ BF16 FMA lanes, so A_{VU} scales with N_{VU}.
- **On-chip SRAM (A_{VMEM}):** Scales with `vmem_size_MB`, accounting for TSMC N5 bit-cell area plus peripheral overhead (sense amplifiers, decoders, ECC).
- **ICI SerDes (A_{ICI}):** Scales with `ici_bw_GBps`. The SerDes converts parallel on-chip data into high-speed serial signals for multi-chip communication; more bandwidth requires more physical lanes.
- **HBM Controllers (A_{HBM}):** Scales with `hbm_bw_GBps` (digital logic only; the HBM PHY resides on the HBM base die, not the accelerator die). More bandwidth requires more memory channels and wider internal data paths.

The fixed additive constants are:

[5] https://raw.githubusercontent.com/MichaelWang237/ece511-2026-public/refs/heads/main/mp4/feasibility.py

- **Control logic (A_{ctrl}):** A fixed area for blocks that are present on every chip regardless of compute configuration: PCIe controller, power management unit, JTAG/debug logic, fuse arrays, thermal sensors, ICI router digital logic, and chip-level boot circuitry.
- **Unmodeled microarchitecture ($A_{unmodeled}$):** A fixed estimate for features present in current-generation TPUs that NeuSim does not simulate, including weight/activation caches (CMEM, SPMEM) and sparse cores.

Reticle constraint: Each individual chip must satisfy $A_{total} \leq 858$ mm^2, the approximate reticle field limit for TSMC N5.

How to Use the Area Model

The area model is provided in `area_model.py` at the root of the repository. Given a `ChipConfig`, call `estimate_area()` to get an `AreaBreakdownMM2` object with per-component area and the total:

```
import json
from neusim.configs.chips.ChipConfig import ChipConfig
from feasibility import estimate_area, estimate_perimeter_usage, estimate_cost

chip = ChipConfig(**json.load(open("configs/chips/tpuv5p.json")))
area = estimate_area(chip)

print(area.total_mm2)      # total die area
print(area.sa_mm2)         # SA contribution
print(area.vmem_mm2)       # VMEM SRAM contribution
# ... etc.
```

To check the reticle constraint:

```
assert area.total_mm2 <= 858, f"Die too large: {area.total_mm2:.1f} mm2"
```

You can also modify `ChipConfig` fields directly to explore design points without creating a new JSON file:

```
chip = ChipConfig(num_sa=4, sa_dim=256, num_vu=4, vmem_size_MB=128,
                  hbm_bw_GBps=3200, hbm_size_GB=96, ici_bw_GBps=400)
area = estimate_area(chip)
```

Perimeter and I/O Constraint Model

Modern accelerators are constrained in how much data they can push in and out of the chip per unit time—a limit dictated by the physical perimeter of the silicon die. To maximize the available perimeter while remaining fabricable within a single reticle field, we assume a best-case 2:1 (width-to-height) aspect ratio for your chip. Given your calculated A_{total}, the available die perimeter P_{die} is:

$$P_{die} = 2\left(\sqrt{2 \cdot A_{total}} + \sqrt{\frac{A_{total}}{2}}\right)$$

Your I/O bandwidth (HBM and ICI) consumes physical edge length:

- **HBM Edge:** The number of pins needed to realize each stack's bandwidth require a proportional amount of die-edge length to be available on the logic die.
- **ICI Edge:** Each bidirectional SerDes lane consumes a fixed amount of die-edge length determined by the number of C4 bumps and their pitch.

Constraint: For your chip to be physically viable, the total consumed edge length must be less than or equal to $0.7 \cdot P_{die}$. The remaining 30% of the perimeter is reserved for power delivery bumps, PCIe, and other chip-level I/O.

How to Use the Perimeter Model

Call `estimate_perimeter_usage()` with your chip config and the area breakdown:

```
perimeter = estimate_perimeter_usage(chip, area)

print(perimeter.die_perimeter_mm)     # total die perimeter
print(perimeter.usable_perimeter_mm)  # 70% available for HBM + ICI
print(perimeter.hbm_edge_mm)          # edge consumed by HBM
print(perimeter.ici_edge_mm)          # edge consumed by ICI
print(perimeter.utilization)          # fraction used (must be <= 1.0)
print(perimeter.is_feasible)          # True if design passes
```

Cost Model

Your design budget is restricted to $450,000 across your entire serving system (all chips combined). The cost model accounts for three primary expenses:

$$C_{total} = C_{die} + C_{HBM} + C_{ICI}$$

- **Die fabrication (C_{die}):** Scales with total die area A_{total}. Reflects TSMC N5 wafer cost amortized over usable die area, including packaging and test overhead.
- **HBM modules (C_{HBM}):** Scales with HBM capacity (`hbm_size_GB`), reflecting HBM3/HBM3e module pricing.
- **ICI fabric (C_{ICI}):** Scales with ICI bandwidth (`ici_bw_GBps`), reflecting the cost of cables.

How to Use the Cost Model

```
cost = estimate_cost(chip, area)

print(cost.die_usd)    # fabrication cost for this chip
print(cost.hbm_usd)    # HBM module cost
print(cost.ici_usd)    # ICI link cost
print(cost.total_usd)  # total per-chip cost

# For a multi-chip system with N identical chips:
N = 8
assert N * cost.total_usd <= 450_000, "Over budget"
```

Note that HBM cost is driven by *capacity* (`hbm_size_GB`), while the perimeter constraint is driven by *bandwidth* (`hbm_bw_GBps`). You must ensure both are physically consistent: a reasonable ratio of HBM bandwidth to capacity for HBM3e is roughly 20–45 GB/s per GB (e.g., TPU v5p has 2765 GBps / 95 GB $\approx$ 29 GB/s per GB).

6 Submission Guidelines

6.1 Part II guidelines

- Submit the patch file to the corresponding Gradescope assignment.
- For § 2.2 and § 2.3, place answers to the questions in your report document. You should also include all plots and diagrams as figures. Your answers should refer to these figures where applicable.

6.2 Part III guidelines

- For the artifact, please include the traces produced by the simulations in 3.2. You are not required to produce code for this section, but if you made any important code changes (scripts, etc.), you may include them in the artifact submission as well under a sub-folder named code. The artifact submission should be a single ZIP file.
- Include all other deliverables for this section in the report. All plots and diagrams should be included as figures, and your answers should refer to them when applicable.

6.3 Part IV guidelines

- For the artifact, submit code and simulation traces in a zip file. Place the traces produced by simulations under a subfolder called `/traces`. Include the code changes (including `run_pd_disagg.py`) as a patch file.
- All other deliverables – answers to the questions and any plots/figures – should be included in the report.

6.4 Part V guidelines

- For the artifact, submit code patch files and simulation traces in a zip file. Place the traces produced by simulations under a subfolder called `/traces`.
- All other deliverables – answers to the questions and any plots/figures – should be included in the report.

References

[1] Sandhini Agarwal, Lama Ahmad, Jason Ai, Sam Altman, Andy Applebaum, Edwin Arbus, Rahul K Arora, Yu Bai, Bowen Baker, Haiming Bao, et al. gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925*, 2025.

[2] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020.

[3] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, et al. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th annual international symposium on computer architecture*, pages 1–14, 2023.

[4] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*, pages 1–12, 2017.

[5] Thomas Norrie, Nishant Patil, Doe Hyun Yoon, George Kurian, Sheng Li, James Laudon, Cliff Young, Norman Jouppi, and David Patterson. The design process for google's training chips: Tpuv2 and tpuv3. *IEEE Micro*, 41(2):56–63, 2021.

[6] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. Splitwise: Efficient generative llm inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 118–132. IEEE, 2024.

[7] Ian Schneider, Hui Xu, Stephan Benecke, David Patterson, Keguo Huang, Parthasarathy Ranganathan, and Cooper Elsworth. Life-cycle emissions of ai hardware: A cradle-to-grave approach and generational trends. *arXiv preprint arXiv:2502.01671*, 2025.

[8] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[9] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, et al. Resiliency at scale: Managing {Google's}{TPUv4} machine learning supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 761–774, 2024.